7zpus.swe@gmail.com

Glossario

Versione: 1.0.0
Responsabile: Laoud Zakaria
Destinatari: Sanmarco Informatica S.p.A., Prof. Tullio Vardanega, Prof. Riccardo Cardin, Team 7-ZPUs

---

Indice

A 3

B 4

C 5

D 6

E 7

F 8

G 8

H 9

I 9

J 10

K 11

L 11

M 11

N 12

O 12

P 12

Q 14

R 14

S 15

T 17

U 17

V 18

W 18

X 19

Y 19

Z 19

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
1.0.0	2025/02/17	Laoud Zakaria	Vigolo Davide	Approvazione per baseline RTB
0.4.0	2025/12/28	Vigolo Davide	Soligo Lorenzo	Aggiunta termini Analisi dei Requisiti
0.3.0	2025/12/28	Fattoni Antonio	Vigolo Davide	Aggiunta termini Analisi dei Requisiti
0.2.0	2025/12/28	Soligo Lorenzo	Fattoni Antonio	Aggiunta termini Analisi dei Requisiti
0.1.0	2025/10/31	Vigolo Davide	Gingilino Aaron	Stesura iniziale

A

- **Action**: una GitHub Action è una componente eseguibile (definita tramite metadati action.yml) che implementa una singola funzione di automazione e che può essere richiamata in un workflow GitHub Actions per automatizzare operazioni come unit testing.
- **Agile**: approccio iterativo e incrementale allo sviluppo software che enfatizza la collaborazione, la flessibilità e la consegna continua di valore agli utenti finali. I principi fondamentali dell'Agile includono l'adattabilità ai cambiamenti, la comunicazione diretta tra i membri del team e il coinvolgimento attivo degli stakeholder durante tutto il processo di sviluppo.
- **Aggregazione documentale**: tipologia di documento, rappresenta un insieme di documenti correlati tra loro.
- **AiP**: dall'inglese Archival Information Package, è il fulcro di un sistema di archiviazione OAIS. L'AiP è l'entità dalla quale vengono poi estratte le informazioni che comporranno il DiP. Un DiP può contenere informazioni provenienti da più AiP.
- **Amministratore**: ruolo del team che si occupa della gestione dell'infrastruttura, della configurazione degli ambienti di sviluppo e produzione e degli strumenti di sviluppo.
- **Analisi dei Requisiti**: Fase del processo di sviluppo software in cui vengono raccolti, documentati e analizzati i requisiti, funzionali e non, del sistema da sviluppare. Questa fase è fondamentale per comprendere le esigenze degli utenti e tradurle in un linguaggio tecnico non ambiguo, che permetta e favorisca lo sviluppo successivo del design del sistema software.

- **Analista**: ruolo del team di progetto che si occupa di raccogliere, documentare e analizzare i requisiti del sistema software, interfacciandosi con il committente e gli altri stakeholder per garantire una comprensione chiara e condivisa delle esigenze del progetto.
- **Anteprima**: nell'ambito dell'analisi dei requisiti coincide con la visualizzazione del file specifico.
- **AOO**: Area Organizzativa Omogenea.
- **Architettura**: è la struttura organizzativa di un sistema che identifica e caratterizza le componenti software e le loro interazioni, garantendo la soddisfazione dei requisiti funzionali e non funzionali, come scalabilità, manutenibilità e sicurezza.
- **Artefatti**: prodotti tangibili che vengono creati durante il processo di sviluppo software, come documenti, diagrammi, codice sorgente, test case e altri elementi che rappresentano il lavoro svolto e i risultati ottenuti.
- **AS**: per Assegnatario
- **As Is**: tecnica per l'Analisi dei Requisiti. Prevede di immaginare il mondo senza la risorsa, con le sue conseguenti limitazioni.
- **Attività di Progetto**: azioni che portano allo sviluppo del progetto. Componenti dei Processi di progetto.
- **Attori**: entità che interagiscono con il sistema software.
- **Attore Principale**: colui che attivamente interagisce con il sistema.
- **Attore Secondario**: colui che interagisce in modo passivo, viene invocato dal sistema in aiuto per offrire una funzionalità.

B

- **Baseline**: prodotto che soddisfa i vincoli di una Milestone, non solo per Efficacia ma anche per Efficienza. Una baseline è una versione specifica di un prodotto software che è stata formalmente revisionata e approvata, servendo come punto di riferimento per lo sviluppo futuro. Le baseline sono utilizzate per tracciare le modifiche e garantire la coerenza del progetto nel tempo.
- **Board Scrum**: strumento di gestione del lavoro utilizzato nei progetti Agile, in particolare nella metodologia Scrum, per visualizzare e organizzare le attività del team. La board Scrum è suddivisa in colonne che rappresentano le diverse fasi del processo di sviluppo, come "To Do", "In Progress" e "Done". Le attività vengono

rappresentate come card che possono essere spostate da una colonna all'altra man mano che il lavoro procede.

- **Branch**: un branch (o ramo) in Git è una versione parallela del codice sorgente che consente di lavorare su funzionalità o correzioni senza influenzare il ramo principale (di solito chiamato "main" o "master"). I branch possono essere uniti (merged) per incorporare le modifiche nel ramo principale.
- **Brainstorming**: attività che aiuta a estrarre i requisiti. Necessita di un metodo, una struttura organizzata e quindi è una Cerimonia. È quindi una discussione tra pari che, a partire da una agenda chiara e vigilata da un Maestro di Cerimonia, permette la libera espressione delle idee. Ogni membro però ha un tempo limitato per esprimere le proprie idee. È inoltre necessaria la figura di colui che colleziona i punti salienti che emergono dalla discussione. Risultano utili le Mappe Mentali per schematizzare in modo più comodo i punti di un BrainStorming.
- **Build**: processo di compilazione del codice sorgente e di creazione di un pacchetto eseguibile o distribuibile del software. Il processo di build include la compilazione del codice, la risoluzione delle dipendenze, l'esecuzione dei test e la generazione di artefatti come file eseguibili, librerie o pacchetti.

C

- **Campi comuni**: campi che sono presenti in qualsiasi tipo di documento all'interno del DiP.
- **Canale di attivazione**: canale attraverso cui un processo (di conservazione) è stato attivato.
- **Canale di terminazione**: canale attraverso cui un processo (di conservazione) è stato terminato.
- **Ciclo di vita**: insieme di fasi che un prodotto software attraversa durante il suo sviluppo, dalla concezione alla distribuzione e manutenzione. Le fasi tipiche del ciclo di vita includono l'analisi dei requisiti, la progettazione, l'implementazione, il testing, la distribuzione e la manutenzione.
- **CI**: Continuous Inspection, pratica di sviluppo software che prevede l'analisi continua del codice sorgente per identificare difetti, vulnerabilità e problemi di qualità. Essa utilizza strumenti automatizzati per eseguire analisi statiche del codice, fornendo feedback immediato agli sviluppatori e contribuendo a mantenere elevati standard di qualità durante tutto il ciclo di sviluppo.

- **CI**: Continuous Integration, pratica di sviluppo software in cui le modifiche al codice vengono automaticamente testate e integrate nel repository principale tramite l'uso di strumenti di automazione come GitHub Actions. Questo processo aiuta a identificare e risolvere rapidamente i problemi, migliorando la qualità del software.
- **CIs**: Configuration Items, elementi di configurazione che rappresentano componenti software, hardware o documentali che devono essere gestiti e controllati durante il processo di sviluppo e manutenzione del software.
- **Classe documentale**: insieme di documenti che condividono caratteristiche comuni (e.g. "Fatture").
- **Classificazione dei Requisiti**: organizzazione dei requisiti a "cassetti". Un esempio è la classificazione ad albero, distinguendo tra requisiti di Prodotto (System Requirement), sui Processi (Process Requirement) e sul Contratto (Project Requirement).
- **Commit**: un'operazione in Git che salva le modifiche apportate ai file nel repository locale. Ogni commit rappresenta un'istantanea del progetto in un determinato momento e include un messaggio descrittivo delle modifiche effettuate.
- **Committente**: lo stakeholder che acquisisce o procura il prodotto o servizio dal fornitore, finanzia il progetto e ha l'ultima parola sulla validazione (collaudo) per accettare il prodotto finale.
- **Conservazione**: nell'ambito OAIS, la conservazione è il processo di mantenimento a lungo termine dei contenuti digitali, garantendo che rimangano accessibili e utilizzabili nel tempo.
- **Consuntivo di Progetto**: elenco quantitativo di ciò che è stato fatto nel periodo.
- **Crawler**: programma software che esplora automaticamente le pagine web o i documenti digitali per raccogliere informazioni, indicizzare contenuti o estrarre dati specifici.
- **Cruscotto di qualità**: strumento visivo che fornisce una panoramica delle metriche di qualità del software e dei processi di sviluppo, consentendo ai team di monitorare e analizzare le prestazioni del progetto in tempo reale al termine di ogni Sprint.
- **Custom metadata**: metadati specificati dall'implementazione specifica dello standard OAIS, e dunque non regolati da esso.

D

- **DAI**: Documento Amministrativo Informatico

- **Dashboard**: strumento visivo che fornisce una panoramica delle metriche di progetto, consentendo ai team di monitorare e analizzare le prestazioni del progetto in tempo reale al termine di ogni Sprint.
- **Dati di registrazione**: dati che descrivono le informazioni riguardanti la registrazione di un documento comprendenti la tipologia di flusso (e.g. "Uscita") e la tipologia di registro (e.g. "protocollo ordinario").
- **Dati di verifica**: insieme di informazioni che rappresentano lo stato di verifica del documento quali: "firmato digitalmente", "sigillato elettronicamente", "marcato temporalmente", "conformità copie immagine su supporto informatico".
- **Decisioni**: scelte prese durante il processo di sviluppo del progetto, che possono concretizzarsi in aspetti tecnici, organizzativi o di gestione del progetto. Le decisioni sono spesso documentate per garantire la tracciabilità e la comprensione delle scelte fatte.
- **DI**: Documento Informatico
- **Diagrammi UML**: diagramma rappresentato in linguaggio Unified Modeling Language, utilizzato per visualizzare, specificare, costruire e documentare i componenti di un sistema software. I diagrammi UML includono diagrammi delle classi, diagrammi di sequenza, diagrammi dei casi d'uso e altri tipi di diagrammi che aiutano a rappresentare le diverse prospettive del sistema.
- **DIP**: Dissemination Information Package, pacchetto informativo che rispetta lo standard OAIS (Open Archival Information System) per la conservazione digitale. Il DIP contiene dati e informazioni estratte dall'archivio digitale, organizzati per essere distribuiti a utenti o sistemi esterni.
- **DIPReader**: applicativo software che permette la visualizzazione e l'interazione con i DIP, il prodotto finale del progetto.
- **Documento**: aggregato di file, rispettivamente uno principale e zero o più allegati.
- **Documento Primario**: documento che è identificato come primario all'interno di un insieme di documenti collegati.
- **Driver**: componente software che consente al sistema operativo di comunicare con un dispositivo hardware, come una stampante.

E

- **Efficacia**: raggiungere gli obiettivi prefissati.

- **Efficienza**: raggiungere gli obiettivi consumando solo le risorse previste per tali obiettivi.
- **Epic**: tipologia di work item Jira che rappresenta un obiettivo di alto livello.
- **Estensione**: componente software che estende le funzionalità di un'applicazione, come ad esempio un'estensione del browser o un plugin per un IDE.
- **Estensioni**: nel contesto dei casi d'uso, rappresentano i percorsi alternativi o le variazioni rispetto al percorso principale (Happy Path) che possono verificarsi durante l'esecuzione di un caso d'uso. Le estensioni descrivono scenari che si discostano dal flusso principale, come errori, eccezioni o condizioni particolari che possono influenzare il comportamento del sistema.

F

- **Fascicolo**: aggregazione documentale informatica strutturata e univocamente identificata contenente atti, documenti o dati informatici prodotti e funzionali all'esercizio di una attività o allo svolgimento di uno specifico procedimento.
- **Feature**: funzionalità o caratteristica specifica di un sistema software che fornisce valore agli utenti finali. Le feature possono essere descritte in termini di requisiti funzionali e sono spesso suddivise in task più piccoli.
- **Filtro**: meccanismo che consente di restringere i risultati di una ricerca in base a criteri specifici, valorizzati dall'utente.
- **Firma digitale**: parte di un documento che consente di identificare in modo univoco il firmatario e di garantire l'autenticità e il non ripudio del documento stesso.
- **Formato**: specifica tecnica che definisce la struttura e la rappresentazione dei dati all'interno di un documento o di un file.
- **Fornitore**: il soggetto che fornisce un prodotto o servizio al committente, responsabile della realizzazione del progetto secondo i requisiti stabiliti.
- **Framework**: è un'architettura software predefinita che fornisce strumenti, librerie e linee guida per strutturare e sviluppare applicazioni in modo rapido ed efficiente.

G

- **Garanzia della qualità**: insieme di attività pianificate e sistematiche implementate all'interno di un sistema di qualità per fornire fiducia che un prodotto o servizio soddisferà i requisiti di qualità specificati. La garanzia della qualità si concentra

sulla prevenzione dei difetti attraverso processi ben definiti, standard e procedure, piuttosto che sulla rilevazione dei difetti dopo che si sono verificati.

- **Generalizzazioni**: nel contesto dei casi d'uso, rappresentano il concetto di ereditarietà tra caso d'uso padre e caso d'uso figlio. Il caso d'uso figlio eredita tutte le caratteristiche del caso d'uso padre, ma può anche aggiungere funzionalità specifiche o modificare il comportamento ereditato.
- **Git**: sistema di controllo di versione distribuito, utilizzato per tracciare le modifiche nel codice sorgente durante lo sviluppo del software. Permette a più sviluppatori di collaborare su un progetto in modo efficiente.
- **GitHub**: piattaforma di hosting per il controllo di versione e la collaborazione che utilizza Git. Consente agli sviluppatori di ospitare i loro repository Git online, facilitando la condivisione del codice, la gestione delle versioni e la collaborazione tra team.
- **Good Practice**: una buona pratica è una metodologia o tecnica riconosciuta come efficace e raccomandata, basata sull'esperienza e sui risultati ottenuti in progetti precedenti.

H

- **Happy Path**: percorso principale o scenario ideale in cui tutto funziona come previsto durante l'esecuzione di un caso d'uso, senza incontrare errori o eccezioni.
- **Hotfix**: una correzione rapida e urgente applicata a un software per risolvere un bug critico o un problema di sicurezza, spesso senza seguire il normale processo di sviluppo e test.

I

- **Id aggregazione primario**: identificativo univoco e persistente del livello superiore di fascicolazione nel caso in cui si stia definendo un sottofascicolo o una sottoserie.
- **IDE**: Integrated Development Environment, ambiente di sviluppo integrato che fornisce strumenti e funzionalità per facilitare la scrittura, la compilazione, il debug e la gestione del codice sorgente durante lo sviluppo software, ad esempio Visual Studio Code, Eclipse o IntelliJ IDEA.
- **Identificativo di formato**: sequenza di caratteri che specifica il formato del documento in questione, ad esempio "PDF", "DOCX", "JPEG", ecc.

- **Inclusioni**: nel contesto dei casi d'uso, rappresentano i percorsi che vengono sempre eseguiti in aggiunta al percorso principale (Happy Path) durante l'esecuzione di un caso d'uso. Le inclusioni descrivono funzionalità o comportamenti che sono parte integrante del flusso principale e che devono essere eseguiti ogni volta che il caso d'uso viene eseguito.
- **Indice di classificazione**: codifica del documento secondo il Piano di classificazione utilizzato
- **Indicizzazione**: processo di catalogazione e organizzazione degli elementi presenti all'interno del DiP, predisponendoli per una ricerca efficiente.
- **Indirizzi digitali**: informazioni di contatto digitale come email o altri canali di comunicazioni.
- **Infrastruttura**: insieme di risorse hardware, software e di rete necessarie per supportare lo sviluppo, la distribuzione e la manutenzione di un sistema software.
- **Ingegneria del Software**: approccio sistematico, disciplinato e quantificabile allo sviluppo, all'uso, alla manutenzione e al ritiro del software.
- **Integrità**: proprietà che garantisce che i dati non vengano alterati o corrotti durante la loro elaborazione o trasmissione.
- **IPA**: Indice delle Pubbliche Amministrazioni
- **Issue**: problema, bug o richiesta di funzionalità presente e tracciabile in un sistema di Issue Tracking. Un commit può essere associato a una o più issue.
- **Issue Tracking System (ITS)**: sistema software utilizzato per gestire e tracciare i problemi, i bug e le richieste di funzionalità durante lo sviluppo del software. Gli ITS aiutano i team di sviluppo a organizzare, assegnare e monitorare lo stato delle issue, migliorando la comunicazione e la collaborazione tra i membri del team.

J

- **Jira**: strumento di gestione di progetto sviluppato da Atlassian, utilizzato principalmente come Issue Tracking System (ITS), la gestione dei progetti Agile e la pianificazione delle attività di sviluppo software.
- **JQL**: Jira Query Language, un linguaggio di query utilizzato in Jira per filtrare e cercare issue in base a criteri specifici, consentendo agli utenti di creare query personalizzate per visualizzare solo le issue che soddisfano determinate condizioni.

K

L

- **LaTeX**: sistema di composizione tipografica utilizzato per la scrittura di documenti scientifici, tecnici e accademici, che consente di creare documenti di alta qualità con una formattazione precisa e professionale.
- **LaTeX Workshop**: estensione per Visual Studio Code che fornisce un ambiente completo per la scrittura, la compilazione e la gestione di documenti LaTeX, offrendo funzionalità come l'evidenziazione della sintassi, il completamento automatico, la visualizzazione in tempo reale e il supporto per la gestione delle bibliografie.
- **LLM**: Large Language Model.

M

- **Marcatura temporale**: attributo rappresentato tramite timestamp.
- **Merge**: l'operazione di unione di due rami (branch) in Git. Quando si esegue un merge, le modifiche apportate in un ramo vengono integrate nel ramo di destinazione, combinando il lavoro di più sviluppatori.
- **Metadati**: dati che descrivono altri dati, nel caso dei pacchetti DiP, i metadati descrivono gli elementi contenuti in esso e sono quindi di primario interesse.
- **Metriche**: strumenti di misurazione quantitativa che forniscono informazioni oggettive sullo stato e sulle prestazioni di un progetto. Le metriche aiutano a monitorare l'avanzamento, identificare problemi e prendere decisioni informate durante lo sviluppo del software.
- **Milestone**: punti nel tempo e nelle risorse che traccia l'avanzamento del progetto, posizionate a partire dagli obiettivi finali, incrementali ma per piccoli passi, quantitativi e misurabili. Un punto di riferimento significativo in un progetto di sviluppo software che rappresenta il completamento di una fase importante o il raggiungimento di un obiettivo specifico. I milestone aiutano a monitorare i progressi del progetto e a pianificare le attività future.
- **Mock**: oggetto simulato che imita il comportamento di un componente reale in un sistema software, utilizzato principalmente nei test per isolare e verificare il funzionamento di specifiche parti del codice senza dipendere da componenti esterni o complessi.

N

O

P

- **PAE**: per le Amministrazioni Pubbliche Estere
- **PAI**: per le Amministrazioni Pubbliche Italiane
- **Partita IVA**: codice identificativo univoco per le imprese e i professionisti in Italia, utilizzato per scopi fiscali e amministrativi.
- **PB**: (Product Baseline) un'istantanea (snapshot) del progetto al termine delle attività di pianificazione con il progetto approvato dal committente.
- **PF**: per Persona Fisica
- **PG**: per Organizzazione
- **Piano di classificazione**: URI del Piano di classificazione pubblicato.
- **Piano di Progetto**: documento che descrive l'organizzazione del progetto, le attività da svolgere, le risorse necessarie e i tempi previsti per il completamento del progetto.
- **Piano di Qualifica**: organizzazione di attività di V&V durante lo sviluppo, periodico per arrivare alla Validazione finale garantendo la Validità di tutto il lavoro (Lavorare sempre nel modo giusto per essere nel giusto).
- **Pipeline di CI/CD**: insieme di processi automatizzati che consentono di integrare, testare e distribuire il software in modo continuo, migliorando l'efficienza e la qualità del processo di sviluppo. (CI: Continuous Integration, CD: Continuous Deployment/Delivery)
- **Plan-Do-Check-Act**: ciclo di miglioramento continuo che prevede quattro fasi: Pianificare (Plan), Eseguire (Do), Verificare (Check) e Agire (Act). Questo ciclo aiuta a identificare e risolvere problemi, migliorare processi e garantire la qualità del prodotto.
- **PostCondizioni**: stato in cui si deve trovare il sistema dopo che l'attore ha utilizzato una funzionalità.
- **Precondizioni**: stato in cui si deve trovare il sistema per permettere all'attore di compiere un'azione.

- **Procedimento amministrativo**: metadato funzionale volto ad indicare il procedimento a cui il fascicolo afferisce, nonché lo stato di avanzamento e le relative fasi.
- **Processo**: insieme di attività correlate e coese che trasformano ingressi (bisogni) in uscite (prodotti) secondo regole date, consumando risorse nel farlo.
- **Processo di conservazione**: nel nostro caso specifico, ogni AiP ha un processo di conservazione di riferimento. Questo processo permette di tracciare il ciclo di vita e la conservazione dell'AiP nel sistema di conservazione.
- **Procedure**: insieme di regole e linee guida che definiscono come eseguire determinate attività o processi all'interno di un progetto, garantendo coerenza e qualità nel lavoro svolto.
- **Product Baseline (PB)**: è lo stato di avanzamento che definisce il prodotto in una versione stabile, verificata e potenzialmente rilasciabile (o comunque utilizzabile). Rappresenta la concretizzazione dell'architettura ipotizzata nella fase precedente (RTB).
- **Progettazione**: fase del processo di sviluppo software in cui vengono definiti l'architettura, i componenti e le interfacce del sistema software, basandosi sui requisiti raccolti durante l'analisi dei requisiti.
- **Progettista**: ruolo del team che si occupa della progettazione (vedi "Progettazione")
- **Progetto**: insieme di attività che:
  - Devono raggiungere dati obiettivi a partire da date specifiche
  - Hanno un inizio e una fine fissate in calendario
  - Dispongono di risorse limitate (persone, tempo, denaro, strumenti)
  - Consumano risorse nel loro svolgersi
- **Programmatore**: ruolo del team che si occupa della realizzazione del software (implementazione) e dei test automatici.
- **Proof of Concept**: dimostrazione pratica che una determinata idea, tecnologia o approccio è fattibile e può essere implementato con successo. Nel contesto dello sviluppo software, un proof of concept viene spesso utilizzato per testare l'efficacia di una soluzione tecnica o per validare un'ipotesi prima di investire risorse significative nello sviluppo completo del progetto.
- **Proponente**: è colui che esprime le aspettative iniziali ("requisiti utente") che il sistema deve soddisfare. Il proponente soddisfa le condizioni di aver posto i

requisiti e aver generato l'opportunità, ma non è necessariamente l'utente finale che userà il software.

- **Pull request (PR)**: una richiesta di pull è una funzionalità di GitHub che consente agli sviluppatori di proporre modifiche al codice sorgente di un progetto. Quando un sviluppatore crea una pull request, sta chiedendo al team di revisione del progetto di esaminare le modifiche apportate e, se approvate, di unirle (merging) nel ramo principale del repository.
- **Push**: un'operazione in Git che invia le modifiche apportate al repository locale a un repository remoto, rendendo le modifiche disponibili per altri sviluppatori che collaborano al progetto.

Q

- **Quality gates**: in SonarQube sono un insieme di condizioni basate su metriche (es. copertura test, bug, vulnerabilità) che determinano se un progetto soddisfa gli standard di qualità definiti prima del rilascio. Funzionano come un "semaforo" (Passed/Failed) automatico, bloccando il codice non idoneo.
- **Query**: nell'ambito di ricerca semantica, descrizione delle informazioni che si vogliono ottenere formulata in linguaggio naturale (e.g. "fatture salotto").

R

- **Report**: documento che riporta i risultati di un'attività o di un processo, spesso accompagnato da analisi e commenti.
- **Repository**: un repository è un archivio centralizzato in cui viene memorizzato il codice sorgente di un progetto software, insieme alla sua cronologia delle modifiche. I repository sono utilizzati per gestire le versioni del codice, facilitare la collaborazione tra sviluppatori e mantenere una traccia delle modifiche apportate nel tempo.
- **Requirements and Technology Baseline (RTB)**: è lo stato di avanzamento del progetto in cui vengono "congelati" e approvati i requisiti e le scelte tecnologiche fondamentali. Serve a dimostrare la fattibilità del progetto prima di procedere allo sviluppo.
- **Requisiti**: sono le specifiche e le condizioni che un sistema software deve soddisfare per essere considerato completo e funzionante. I requisiti possono essere suddivisi in diverse categorie, come requisiti funzionali (cosa il sistema deve fare) e requisiti non funzionali (come il sistema deve comportarsi, ad esempio in termini di prestazioni, sicurezza, usabilità, ecc.).

- **Requisiti Desiderabili**: grado di urgenza di un requisito che lo etichetta. Non necessari ma con valore aggiunto che può essere avanzato a Obbligatorio se possibile.
- **Requisiti Obbligatori**: grado di urgenza di un requisito che lo etichetta. Irrinunciabili per i committenti. Conviene che sia un numero limitato per evitare di rimanere vincolati.
- **Requisiti Opzionali**: grado di urgenza di un requisito che lo etichetta. Relativamente utili e contrattabili più avanti nel progetto.
- **Responsabile**: ruolo del team che si occupa della gestione del progetto, della pianificazione delle attività e della comunicazione con il committente e gli stakeholder.
- **Retrospettiva**: analisi di ciò che è stato fatto in precedenza con maggiore esperienza e consapevolezza in modo da ottimizzare, crescere in comprensione oltre che nella crescita del prodotto. È ciò che chiude un periodo e prepara al successivo, con l'Ideazione di Misure Correttive e la pianificazione (Consultivo a Finire).
- **Ricerca semantica**: tipo di ricerca che si basa sulla comprensione del significato delle parole e delle frasi, piuttosto che sulla semplice corrispondenza di parole chiave. La ricerca semantica mira a fornire risultati più pertinenti, tenendo conto del contesto e delle relazioni tra i concetti.
- **Rischi**: eventi o condizioni che, se si verificano, possono avere un impatto negativo sugli obiettivi del progetto. La gestione dei rischi include l'identificazione, l'analisi, la pianificazione e la mitigazione dei rischi per minimizzare le conseguenze negative sul progetto.
- **RTB**: (Requirement and Technical Baseline) converge l'Analisi dei Requisiti con la risposta Tecnica. Una volta compresi i vincoli tecnologici emerge la necessità che le tecnologie rispecchiano tali vincoli, pesando la scelta in base all'impatto che tali tecnologie hanno sul design (1° Compito della RTB). L'uso di più tecnologie pretende anche che queste eterogeneità possano convivere in un Proof Of Concept (2° Compito della RTB).
- **RUP**: Responsabile Unico Progetto

S

- **Scenario Alternativo**: insieme di passi che descrive un Path che porta a PostCondizioni diverse da quelle dello Scenario Principale.

- **Scenario Principale**: insieme di passi che descrive l'Happy Path di una funzione (passi per arrivare al risultato).
- **Script**: insieme di comandi o istruzioni che vengono eseguiti in sequenza per automatizzare un processo o un'attività specifica all'interno di un progetto di sviluppo software.
- **Sessione di versamento**: si intende la sessione entro la quale i documenti, che successivamente compongono un AiP, vengono versati nel sistema di conservazione.
- **Sistema**: insieme di componenti interconnessi che lavorano insieme per raggiungere un obiettivo comune. Nel contesto dello sviluppo software, un sistema può essere costituito da hardware, software, processi e persone che collaborano per fornire funzionalità e servizi agli utenti finali.
- **Slack Time**: margine tra due attività (sempre maggiore di zero) che attutisce l'impatto di ritardi, in modo da non propagare il ritardo sulle attività successive. Concetto alla base dei diagrammi PERT. Per la valutazione di questi Slack, è necessaria una Gestione dei Rischi accurata formata da Identificazione, Analisi, Pianificazione, Controllo.
- **Software**: insieme di programmi, procedure e documentazione associata che svolge una funzione specifica o fornisce un servizio.
- **Soggetto**: persona fisica o giuridica che viene riferita nei metadati di un documento per indicarne un ruolo specifico o una relazione ad esso (e.g. "assegnatario").
- **Sprint**: un periodo di tempo definito durante il quale un team di sviluppo software lavora per completare un insieme specifico di attività o obiettivi. Gli sprint sono una componente chiave della metodologia Agile, in particolare di Scrum, e aiutano a organizzare il lavoro in iterazioni brevi e gestibili.
- **Stakeholder**: una persona, gruppo o organizzazione che ha un interesse diretto o indiretto in un progetto o in un sistema software. Gli stakeholder possono includere clienti, utenti finali, sviluppatori, manager e altre parti interessate che influenzano o sono influenzate dal progetto.
- **Stato di avanzamento lavori (SAL)**: misura quantitativa del progresso di un progetto, spesso espressa in percentuale, che indica quanto lavoro è stato completato rispetto al totale previsto.
- **Subtask**: un'unità di lavoro più piccola e specifica all'interno di un task più grande. I subtask aiutano a suddividere il lavoro in parti gestibili e facilitano la pianificazione e l'assegnazione delle attività.

- **SW**: per Software
- **System Requirement**: possiamo suddividere i requisiti di sistema: - Constraint (vincoli legali, fisici, culturali, di design, implementativi, ecc) - Functional Requirement (funzioni da svolgere del prodotto) - Attribute (Performance Requirement e Specific Quality Requirement, aka *-lities*). Questa suddivisione ci permette più agilmente di trovare questi requisiti.

T

- **Tempo di conservazione**: periodo di tempo per il quale il documento è conservato nel sistema di conservazione.
- **Task**: un'unità di lavoro specifica e ben definita all'interno di un progetto di sviluppo software. I task rappresentano attività piccole e gestibili che contribuiscono al completamento di funzionalità o obiettivi più ampi.
- **Test**: attività che verifica che il software funzioni come previsto, spesso e per quanto possibile automatizzata.
- **Test di Sistema**: test che verifica la conformità del sistema alle specifiche, con l'obiettivo di identificare eventuali difetti o problemi prima del rilascio. I test di sistema sono eseguiti in un ambiente che simula le condizioni reali di utilizzo del software.
- **To Be**: tecnica per l'Analisi dei Requisiti. Prevede di immaginare il mondo con la risorsa, valutando i miglioramenti e le nuove prospettive che si aprono.
- **Tracciabilità**: serve che i requisiti abbiano un identificativo unico, che debba anche renderli modificabili in modo semplice, per far fronte a possibili aggiunte e/o cancellazioni.

U

- **UOR**: Unità Organizzativa Responsabile.
- **Use Case**: funzionalità di un sistema, che non descrivono però la loro implementazione.
- **User Story**: tecnica per l'Analisi dei Requisiti che è più immaginativa. È una descrizione breve e semplice di una funzionalità del sistema desiderata dall'utente. La User Story può essere anche interna al gruppo di lavoro.
- **UUID**: Universally Unique Identifier, identificatore univoco globale utilizzato per identificare univocamente risorse all'interno di un sistema software.

V

- **V-Model**: modello di sviluppo software che rappresenta un processo di sviluppo a forma di "V", in cui le fasi di progettazione e implementazione sono parallele alle fasi di test e validazione. Il V-Model enfatizza l'importanza della verifica e della validazione in ogni fase del processo di sviluppo.
- **Validazione**: controllo della soddisfazione delle aspettative (Finale).
- **Verifica**: controllo che lo sviluppo non introduca errori.
- **Verificatori**: ruolo del team che si occupa della verifica.
- **Versamento**: azione di conferimento di un documento ad un sistema di conservazione.
- **Versionamento**: processo di gestione delle modifiche al codice sorgente o ad altri artefatti di un progetto, che consente di tenere traccia delle diverse versioni e delle modifiche apportate nel tempo. Il versionamento è spesso gestito tramite sistemi di controllo di versione come Git.
- **VSCode**: Visual Studio Code, un editor di codice sorgente sviluppato da Microsoft, ampiamente utilizzato per la programmazione e lo sviluppo software grazie alla sua leggerezza, estensibilità e supporto per una vasta gamma di linguaggi di programmazione.

W

- **Waterfall**: approccio sequenziale allo sviluppo software in cui le fasi del processo di sviluppo (analisi dei requisiti, progettazione, implementazione, test e manutenzione) vengono eseguite in ordine, con ogni fase che deve essere completata prima di passare alla successiva.
- **Way of Working**: il come si deve lavorare, gli standard, gli obiettivi di qualità garantiti da strumenti ad hoc.
- **Work item**: un'unità di lavoro specifica e ben definita all'interno di un progetto di sviluppo software, che può rappresentare una funzionalità, un bug da risolvere, un'attività di miglioramento o qualsiasi altra attività necessaria per il completamento del progetto.
- **Workflow**: sequenza di attività o processi che devono essere eseguiti per completare un compito specifico all'interno di un progetto di sviluppo software.

- **Working Copy**: una copia locale di un repository Git che contiene tutti i file e le cartelle del progetto. Gli sviluppatori lavorano sulla working copy per apportare modifiche prima di inviarle al repository remoto.
- **Worst Case Scenario**: scenario peggiore che può verificarsi in un progetto software, in cui tutti i rischi si manifestano contemporaneamente, causando conseguenze negative significative.

X

Y

Z